

AI Use Declaration Form

Course: Introduction to Artificial Intelligence

Lab Title: Lab 0 Part A — Environment Setup & Python for AI Foundations

Student Name: ___Naïma Tahirou Maïyaki

Student ID: ___30252027

GitHub Repository Link: <https://github.com/naimatahiroumaiyaki/ai-course-lab0>

Date Submitted: ___6/5/2026

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	Yes
Estimated percentage of the work influenced by AI	28%
Did you attach evidence of AI use where applicable?	Yes

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
ChatGPT	To understand concepts, debug code, explain NumPy, Pandas, and data visualization tasks, and review my solutions.	I asked ChatGPT to explain programming concepts, break down lab exercises, help identify errors in my code, explain outputs, and review my solutions for Exercises 4–7.	Exercises 4–7, including NumPy operations, linear regression, Pandas data analysis, visualizations, CLV calculation, and age grouping.	I ran all code myself in Jupyter/VS Code, tested outputs, corrected errors identified during execution, modified code based on my understanding, reran cells to verify results, and interpreted the final visualizations

				and analysis before submission.

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name
AI-generated draft	No	N/A
Prompt history	No	N/A
Screenshot of AI interaction	Yes	See appendix
Exported AI conversation	No	N/A
AI-generated code snippet	Yes	See appendix
Revised version showing student input	yes	See appendix

4. Student Declaration

I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	yes
Any use of AI tools has been clearly declared in this form.	yes
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	yes
I reviewed, tested, edited, and improved any AI-generated content before submission.	yes
My AI usage does not exceed 25% of the entire work.	no
I understand that undeclared or excessive AI use may be treated as academic misconduct.	yes

Student Signature: ____NaimaTahirou_____

Date: ____6/5/2026_____

Screenshot of AI interaction and AI-generated code snippet

```
avg_clv = customers.groupby('age_group')['clv'].mean()
avg_clv.plot(
    kind = 'bar ',
    color = 'green ',
)
plt.title('Average CLV by Age Group')
plt.xlabel('Age Group')
plt.ylabel('Average CLV')
plt.show()
```

Traceback (most recent call last)

Cell In[36], line 3

1 avg_clv = customers.groupby('age_group')['clv'].mean()

----> 2 avg_clv.plot(
 3 kind = 'bar ',
 4 color = 'green ',
 5)

File C:\Users\valde\miniconda3\envs\libdata\env\libsite-packages\pandas\plotting_core.py:1118, in PlotAccessor._call__(self, *args, **kwargs)

1118 return plot_backend.plot(self._parent, x=x, y=y, kind=kind, **kwargs)

1117 if kind not in self._all_kinds:

-> 1118 raise ValueError(
1119 f"{kind} is not a valid plot kind valid plot kinds: {self._all_kinds}"
1120)

1121 data = self._parent

1122 if isinstance(data, ABCSeries):



The error is very small, but very common.

Look carefully at your code:

`<>` Python



▶ Exécuter

```
avg_clv.plot(
    kind = 'bar ',
    color = 'green',
)
```



Notice:

```
avg_clv = customers.groupby('age_group')['CLV'].mean()
avg_clv.plot( kind = 'bar ',
              color = 'green',
              )
```

That's actually correct. ✅

You successfully translated:

Average CLV by age group



into:



```
plt.figure(figsize = (10,5))
sns.scatterplot(
    x = 'income',
    y = 'CLV',
    hue = 'age_group')
```

```
plt.xlabel('income')
plt.ylabel('CLV')
plt.title('income vs CLV')
plt.show()
```



This is a very good attempt. You're about 90% there.

Issue 1: Typo

You wrote:

<> Python



▶ Exécuter

```
sns.scatterplot(
```

```
print("Missing Values")
print(df.isnull().sum())
```

[27] ✓ 0.1s

```
...
Missing Values
student_id      0
major           0
year            0
exam_score      0
assignments_completed  0
hours_studied   0
performance     0
dtype: int64
```

is it suppose to be the output?



Actually, **no** — that output is a little suspicious based on the original dataset.

Remember the starter code intentionally created:

Python



Exécuter

```
df.loc[np.random.choice(n_students, 10), 'exam_score'] = np.nan
```

$\text{np.linalg.inv}(X.T @ X) @ X.T @ y$ explain it

Let's make it as simple as possible.

The expression is:

Python



Exécuter

```
np.linalg.inv(X.T @ X) @ X.T @ y
```



Don't try to understand it all at once.

is there a very simple way to do this task



Yes.

For **understanding the assignment**, you do not need to understand every detail of the normal equation derivation.

The simplest way to think about Task 3 is:

X = input/features
y = output/target



Linear regression tries to find the coefficients
that connect X to y.



Task 2: Normalize a random array

```
np.random.seed(42)
random_data = np.random.randn(100, 3)

normalized_data = (
    random_data - random_data.mean(axis=0)
) / random_data.std(axis=0)

print("Column Means:")
print(normalized_data.mean(axis=0))

print("\nColumn Standard Deviations:")
print(normalized_data.std(axis=0))
```

[1] ✓ 0.3s

Column Means:
[-2.60902411e-17 1.55431223e-17 -8.88178420e-18]

Column Standard Deviations:
[1. 1. 1.]

my output is not given 0 for the mean



This is actually **correct**. 👍

The reason you're not seeing exact zeros is because of **floating-point precision**.

You got:



i still do not understand, the rows are 100 right , and we have 3 columns so my logic is that the mean and standard deviation of each column should be 0, and 1 respectively for all the 100 rows



You're very close. The confusion is about **what the mean and standard deviation belong to**.

Let's ignore the 100 rows for a moment and use only 3 rows.

Suppose:

Python



▶ Exécuter

```
data =  
[[10, 20, 30],  
 [20, 30, 40],  
 [30, 40, 50]]
```



T

break it down: # Task 2: Normalize a random array
(5 points)
np.random.seed(42)
random_data = np.random.randn(100, 3)
TODO: Normalize each column to have mean=0 and
std=1



Good. Let's learn the concept, not just the code.

The assignment says:

Python



▶ Exécuter

```
# Task 2: Normalize a random array
```

```
np.random.seed(42)  
random_data = np.random.randn(100, 3)
```

```
# TODO: Normalize each column to have mean=0 and std=1
```



why using it ? because it seems like even without it the code can still run

Excellent question.

You're absolutely right:

</> Python



▶ Exécuter

```
np.random.randn(100, 3)
```

will work perfectly fine without:

</> Python



▶ Exécuter

Step 3: How do we normalize?

The formula is:

x

$z = \frac{x - \mu}{\sigma}$

x 1.2

μ 0.0

σ 1.0

$z = \frac{x - \mu}{\sigma} \approx 1.2$

$\Phi(z) \approx 88.5\%$

where:

- x = original value
- μ = mean
- σ = standard deviation




```
explain: matrix = np.ones((5, 5))  
matrix[1:-1, 1:-1] = 0
```

```
print(matrix)
```



Good. Let's understand it line by line rather than just memorizing it.

Line 1

```
break it down for me: # Task 3: Implement linear  
regression solution using normal equation  
# (10 points)  
# Given X (features) and y (target), compute theta  
#  $\theta = (X^T X)^{-1} X^T y$   
X = np.random.randn(50, 3)  
true_theta = np.array([2.5, -1.2, 3.7])  
y = X @ true_theta + np.random.randn(50) * 0.1  
# TODO: Calculate theta_hat using the normal equation  
# TODO: Print the estimated coefficients and compare with  
true_theta
```



This is probably the hardest part of Exercise 4, so let's ignore the code for a moment and understand the story.

What is Linear Regression?



Suppose you want to predict:

<> Python



▶ Exécuter

```
X = np.random.randn(50, 3)

true_theta = np.array([2.5, -1.2, 3.7])

y = X @ true_theta + np.random.randn(50) * 0.1

theta_hat = np.linalg.inv(X.T @ X) @ X.T @ y

print("Estimated Coefficients:")
print(theta_hat)

print("\nTrue Coefficients:")
print(true_theta)
```

<> Python



▶ Exécuter

```
import seaborn as sns

sns.set_style("whitegrid")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histogram + KDE
sns.histplot(
    data=df,
    x='exam_score',
    kde=True,
    ax=axes[0]
)

axes[0].set_title('Distribution of Exam Scores')
axes[0].set_xlabel('Exam Score')
axes[0].set_ylabel('Frequency')

# Boxplot by major
sns.boxplot(
    data=df,
    x='major',
    y='exam_score',
```

